

File_Deduplication

User Guide

AI-enhanced file deduplication and organization: recursive scanning, SHA256 duplicate detection, 250+ file-type classification, local-LLM fallback classification, semantic tagging, and safe dry-run previews.

Version 0.10.0 • August 2026 • Tim Canady

Contents

1. Introduction
2. Before You Run: Pitfalls and Red Alerts
3. Installation and Setup
4. Running the Application
5. The Web Interface (File Workbench)
6. Execution Modes
7. Command-Line Reference
8. Common Recipes
9. Project Structure
10. Version Management and Releases
11. Git Workflow: Committing and Pushing
12. Tests and Utility Scripts
13. Troubleshooting

1. Introduction

File_Deduplication scans a directory tree, detects duplicate files by SHA256 hash, classifies every file into one of 18+ categories, and plans (then optionally executes) a clean, organized folder structure. All potentially destructive operations are gated behind an explicit **--execute** flag with a confirmation prompt; the default mode is always a safe dry-run preview.

Key capabilities:

- **Atomic package detection** — macOS bundles (.app, .pkg, .dmg) are treated as single units instead of thousands of internal files (18–60x faster).
- **Parallel, resumable hashing** — SHA256 hashing runs on a thread pool (--workers), every hash commits to MySQL immediately, and interrupted runs resume from the cache: unchanged files are never hashed twice.
- **Rule-based classification** — MIME types, extensions, filenames, and directory patterns place files into categories such as document, financial, code, or education.
- **Local LLM fallback** — files no rule can place are classified by a local Ollama model from filename, path, and content. Nothing leaves your machine.
- **Semantic tagging** — path context and (optionally) CLIP image analysis generate searchable tags stored in the database.
- **Intelligent size handling** — very large files can be processed metadata-only, skipping expensive hashing.

2. Before You Run: Pitfalls and Red Alerts

Read this section before the first run on real data. None of these will destroy files — the tool never deletes anything — but several can waste hours, fill a disk, or quietly skip data you expected to be processed.

2.1 Red Alerts

- **RED ALERT: Execution COPIES — it does not move or delete.** Despite the plan being labeled MOVE, `--execute` copies files (`shutil.copy2`) and leaves every source file in place. Two consequences: your originals are always safe, and the destination volume needs free space roughly equal to the total size of the organized files. Check free space before executing. Duplicates are marked and can be excluded from the plan (`--skip-duplicates`), but they are never deleted from the source.
- **RED ALERT: Never put `--base-dir` inside the source tree.** If the destination lives under the source (e.g. scanning `/Volumes/home` with `--base-dir /Volumes/home/Organized`), the next scan will ingest the organized copies as new files — every file appears twice, every re-run doubles the noise. Always point `--base-dir` outside the tree being scanned, or add the destination to `.dedupignore`.
- **RED ALERT: Always dry-run first.** Run without `--execute`, read the proposed tree, and only then re-run with `--execute`. The y/N confirmation prompt is the last gate before files are copied.
- **RED ALERT: Interrupting is only safe with `--use-db`.** With the database enabled, Ctrl+C loses at most the files in flight and a re-run resumes from the cache. Without it, all progress from the run is gone.

2.2 Silent Coverage Gaps

These are by design, but each one means some files are not processed the way you might assume:

- **Hidden files are always skipped.** Anything starting with a dot (`.ssh`, `.config`, dotfiles generally) — and everything inside hidden directories — is never scanned, hashed, or organized.
- **Large files are invisible to duplicate detection.** Files above `--metadata-only-size` are not hashed, so two identical 8GB videos will NOT be flagged as duplicates. Choose the threshold with that trade-off in mind.
- **`.dedupignore` excludes silently.** The project ships ignore patterns (currently 19). Review the file before assuming complete coverage of your volume.
- **Directory names can override content classification.** Files inside directories named `src`, `scripts`, `code`, `backup`, `www`, `software`, `adobe`, `microsoft`, and similar are classified by location (`code` / `application` / `backup` / `web`) and their structure is preserved as-is — a PDF inside `/software/` is treated as part of that application tree, not as a document.
- **Existing destination files are skipped, not overwritten.** If a planned destination path already exists, that file is skipped with a warning. Safe for re-runs, but a stale earlier copy will not be refreshed.

2.3 Operational Cautions

- **Database loss mid-run triggers the circuit breaker.** After 3 consecutive DB failures the run continues without persistence (a dry run still completes), `--execute` is refused, and an in-progress execution stops before its next file move. Work done after the breaker trips is not resumable.
- **Very long paths cannot be cached.** The database path column holds 767 characters; deeper NAS paths fail to persist (logged as warnings) and will re-hash on every run.
- **--llm-classify needs a running Ollama server.** Start it with ``ollama serve`` before the run; otherwise the fallback is skipped (gracefully). Each unique unclassifiable file costs roughly 1–2 seconds of local inference. All content stays on your machine.
- **--gui pauses the run at the end.** The pipeline does not finish until the preview window is closed — avoid `--gui` for unattended runs.
- **First runs on network volumes are slow.** Every byte of every sub-threshold file is read over the network for hashing. Use `--workers 8`, `--metadata-only-size`, and `--use-db` so subsequent runs skip unchanged files. A `--max-files 500` trial run validates the whole pipeline in minutes.

3. Installation and Setup

3.1 Requirements

- Python 3.9 or newer
- MySQL 8.x (only when using **--use-db** and related features)
- Ollama with a pulled model (only for **--llm-classify**)
- Slack webhook URL (only for **--notify slack**)

3.2 Installing Dependencies

```
# Core install (light – no ML libraries)
pip install -r requirements.txt

# Optional: CLIP-based image content analysis (~2GB of ML deps)
pip install -r requirements-ai.txt

# Optional: install as a package, giving you the `dedupe` command
pip install .          # add [ai] for the CLIP extras: pip install .[ai]
```

The application degrades gracefully: without the AI extras, image content tagging is simply skipped; without a reachable Ollama server, LLM classification is skipped.

3.3 The .env File

Create a `.env` file in the project root (never commit it — it is gitignored):

```
# MySQL connection (used by core/db.py when --use-db is passed)
DB_HOST=localhost
DB_PORT=3306
DB_NAME=File_Deduplication
DB_USER=your-db-user
DB_PASSWORD=your-db-password

# Optional: Slack notifications
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/...

# Optional: local LLM classification fallback (--llm-classify)
OLLAMA_HOST=http://localhost:11434
LLM_MODEL=llama3.1:8b
```

Database bootstrap: run the scripts in `database/schema/` first (table creation), then any needed `database/migrations/` in order.

4. Running the Application

Three equivalent ways to invoke the same command-line interface:

```
python main.py SOURCE --base-dir DEST      # root wrapper script
python -m core.main SOURCE --base-dir DEST # module form
dedupe SOURCE --base-dir DEST              # console script (after pip install .)
```

SOURCE is the directory tree to scan. **--base-dir DEST** is where the organized structure will be created. Both are required.

4.1 Pipeline Stages

Every run flows through the same stages, in order:

- 1. Scan** — recursively collect files, honoring `--filter`, `--file-types`, `--max-files`, and `.dedupignore` patterns; atomic packages are collapsed into single units. On walks longer than 10 seconds a progress heartbeat logs files matched, directories visited, and the current location.
- 2. Hash** — SHA256 each file on a thread pool sized for the machine (override with `--workers`), or record metadata only above the `--metadata-only-size` threshold. With `--use-db` every hash commits immediately and files unchanged since a previous run are skipped via the cache.
- 3. Deduplicate** — group files by hash; mark duplicates and optionally report or drop them.
- 4. Classify** — rule-based category assignment across a pool of worker processes, then an optional local-LLM pass for whatever the rules left unresolved (`--llm-classify`), then an optional cost-capped cloud pass for the remainder (`--cloud-classify`).
- 5. Tag / Analyze** — with `--use-db`: semantic path tags; optionally image metadata (`--analyze-images`) and CLIP content tags (`--ai-tagging`).
- 6. Plan** — compute the target folder structure (year / type / owner grouping, structure-preserving categories).
- 7. Preview / Execute** — print the plan (and optionally log, notify, or show a GUI); copy files into the destination only with `--execute` after a y/N confirmation. Sources are never removed.

4.2 How the Work Is Parallelised

There is no single worker count, because each stage is limited by a different resource: scanning and hashing wait on disk, the classification and tagging stages are CPU-bound and would serialise on Python's GIL if they used threads, the local-LLM stage is limited by the Ollama server, and image content analysis shares one GPU. Each stage is sized independently. To see the sizing for your machine:

```
python main.py --show-parallelism
```

Two points are worth understanding before tuning anything. First, a process pool is not free: worker startup and the cost of moving each item across a process boundary are real, and on cheap per-file

work a pool is *slower* than staying serial. The application therefore measures a short serial sample of each CPU stage and only builds a pool when the measured cost justifies it. A stage reported as running serially is usually doing the right thing.

Second, the local-LLM stage is bounded by Ollama, not by your core count. Client concurrency is read from **OLLAMA_NUM_PARALLEL**; if that is unset, Ollama serves only a few requests at a time and adding client threads changes nothing. Raise it to use more of the machine:

```
export OLLAMA_NUM_PARALLEL=8
```

Any stage can be overridden with **WORKBENCH_<STAGE>_WORKERS** — for example **WORKBENCH_HASH_WORKERS=32**. Setting one to 1 forces that stage serial, which is the quickest way to isolate a stage while debugging.

5. The Web Interface (File Workbench)

Everything the CLI does can be driven from a browser instead. The web interface, **File Workbench**, runs the same pipeline code as the command line — `core/main.py` and the server both call `core/pipeline.py` — so the two cannot produce different results.

Start the server:

```
./venv/bin/python -m server.app
```

It prints the URL it chose (the first free port from 8000) and binds to localhost only, so it is not reachable from other machines.

5.1 The Four Screens

Screen	Purpose
Run	Choose source and destination, set options, and watch each stage report live throughput and an estimated time remaining. A run can be cancelled at any point; work already completed is kept.
Duplicates	Duplicate groups by content hash, ordered by reclaimable space, with the copy that would be kept marked.
Plan	Every proposed operation, paginated and filterable by path or category. This is the review step: nothing has been written to disk yet.
Jobs	Every run from this session. Reopen one to review its plan or execute it later.

5.2 Execution Is a Copy, and Requires Confirmation

The Execute step **copies** files into the planned destination. Your source tree is left exactly as it was, so a run is undone by deleting the destination directory. It still requires typing a confirmation phrase before anything is written, existing destination files are skipped rather than overwritten, and with the database enabled every operation is logged. If the database becomes unreachable mid-run, execution is refused rather than performed unrecorded.

5.3 Where Job Output Goes

Each run writes its plan, duplicate groups, and result summary to `.workbench/jobs/<job-id>/`. These are run artifacts rather than source, and are excluded from version control. They persist across restarts, so a plan can be reviewed and executed later.

6. Execution Modes

6.1 Dry Run (default)

With no mode flags, the tool scans, hashes, classifies, and prints the proposed folder tree without touching a single file. Always start here.

```
python main.py /Volumes/home --base-dir /organized
```

6.2 Logged Dry Run

Add **--dry-run-log** to write the plan to a timestamped `dry_run_preview_*.json` (or `.txt` with **--log-format txt**) for later review or diffing between runs.

```
python main.py /Volumes/home --base-dir /organized --dry-run-log --log-format json
```

6.3 GUI Preview

Add **--gui** to review the plan in a PySimpleGUI window instead of reading terminal output.

6.4 Execute

Add **--execute** to apply the plan. The preview is still shown first, and you must confirm with **y** at the prompt before any file moves.

```
python main.py /Volumes/home --base-dir /organized --use-db --execute
```

6.5 Interrupting and Resuming

With **--use-db**, long runs are safe to interrupt. Every completed hash is committed to MySQL immediately, so pressing Ctrl+C during the hashing stage loses at most the handful of files in flight. The run exits cleanly with a message confirming how much work was persisted. To resume, simply re-run the same command: files whose path and modification time match a cached entry are skipped without reading a byte (marked "(cached)" in the log), and the run picks up where it left off. Without **--use-db**, interrupted work is not persisted.

If the **database itself** becomes unreachable mid-run, a circuit breaker trips after three consecutive failures: analysis phases continue without persistence (so a dry run still completes), but **--execute** is refused and an in-progress execution stops cleanly before its next file move — file operations are never performed unlogged.

7. Command-Line Reference

7.1 Required Arguments

source (positional)

Root directory to scan. Must exist and be readable.

```
python main.py /Volumes/home --base-dir /organized
```

--base-dir PATH (required)

Base output directory where the organized folder structure is planned and (with --execute) created. Created automatically if the parent directory exists and is writable.

7.2 Scan Control

--filter PATTERN [PATTERN ...]

Only include top-level directories whose names match one of the given patterns. Useful for picking specific user folders out of a large volume.

```
python main.py /Volumes/home --base-dir /organized --filter canadytw canamac
```

--file-types GROUPS

Restrict the scan to file-type groups (comma-separated). Hierarchical groups are supported: 'media' expands to images, videos, and audio.

```
python main.py /photos --base-dir /organized --file-types images,videos
```

--list-file-types

Print all available file-type groups and their extensions, then exit. No other arguments are required.

```
python main.py --list-file-types
```

--max-files N

Stop after processing N files. Ideal for a quick trial run on a very large volume before committing to a full scan.

```
python main.py /Volumes/home --base-dir /tmp/preview --max-files 500
```

--ignore-errors

Skip files that cannot be read (permissions, broken links) instead of aborting the run.

7.3 Hashing and Duplicates

--metadata-only-size SIZE

Files larger than SIZE are recorded metadata-only — no hashing, no duplicate detection. Accepts B, KB, MB, GB, TB. Use this to keep runs fast on volumes full of video files or disk images.

```
python main.py /Videos --base-dir /organized --metadata-only-size 1GB
```

--workers N (default: sized for this machine)

Number of parallel hashing threads. The default is derived from the machine rather than fixed at 4. This flag now affects hashing only: the CPU-bound stages size themselves per stage (see --show-parallelism), so there is no single number to tune.

```
python main.py /Volumes/home --base-dir /organized --use-db --workers 32
```

--show-parallelism

Print how each stage will be executed on this machine — threads, processes, or serial, and how many workers — then exit. Use this before a long run to confirm the sizing, and to see which environment variable overrides each stage.

```
python main.py --show-parallelism
```

--batch-size N (default: 500)

Files per hashing batch. Every completed file is committed to the database immediately (with --use-db), so an interrupted run (Ctrl+C) loses at most the files in flight; batches add periodic checkpoint summaries to the log. Re-running the same command after an interruption resumes from the cache — unchanged files are skipped without re-reading.

--skip-duplicates

Drop duplicate files from the plan entirely; only unique files are organized. Without this flag, duplicates are kept and marked.

--duplicate-report FILE

Write a report of all detected duplicate groups to FILE.

```
python main.py /Volumes/home --base-dir /organized --duplicate-report dupes.txt
```

7.4 Database and AI Features

--use-db

Enable MySQL persistence: hash caching across runs, saved classifications with confidence scores, and the unified file_tags table. Requires the DB_* variables in .env. Semantic path tagging runs automatically when this flag is present.

--llm-classify

Local LLM fallback: files that land in the 'other' category after every rule-based tier are classified by a local Ollama model using filename, path, and a 1KB content snippet for text-like files. Results are cached per content hash (duplicates cost one call). Skipped gracefully if the server is unreachable. Configure via OLLAMA_HOST and LLM_MODEL in .env.

```
python main.py /Volumes/home --base-dir /organized --use-db --llm-classify
```

--analyze-images (requires --use-db)

Extract EXIF, IPTC, and GPS metadata from images into the database.

```
python main.py /photos --base-dir /organized --use-db --analyze-images
```

--ai-tagging (requires `--use-db` and `requirements-ai.txt`)

CLIP-based image content tagging: objects, scenes, and people are identified and stored as keywords and file tags. Run `--analyze-images` first (or in the same invocation) so base metadata exists.

```
python main.py /photos --base-dir /organized --use-db --analyze-images --ai-tagging
```

--cloud-classify

Escalate files that neither the rules nor the local LLM could place to Claude. This is the only tier that costs money, so it runs last and only on the leftovers. Requires `ANTHROPIC_API_KEY` (or an ``ant auth login`` profile); without a credential the tier disables itself and the run continues on the free tiers.

```
python main.py /docs --base-dir /organized --use-db --llm-classify --cloud-classify
```

--cloud-cost-limit USD (default: 1.00)

Hard spend ceiling for `--cloud-classify`. The batch is priced before the first request and skipped entirely if the estimate exceeds this figure; during the run the ceiling is enforced against actual token usage, not the estimate.

```
python main.py /docs --base-dir /organized --cloud-classify --cloud-cost-limit 5.00
```

--cloud-model MODEL

Model used by `--cloud-classify`. Defaults to `claude-opus-5`; also honours the `CLOUD_MODEL` environment variable.

```
python main.py /docs --base-dir /organized --cloud-classify --cloud-model claude-haiku-4-5
```

7.5 Output, Notification, and Execution

--dry-run-log

Write the preview plan to a timestamped `dry_run_preview_*` file.

--log-format json | txt (default: json)

Format for the dry-run log file.

--gui

Show the plan in a PySimpleGUI preview window.

--notify slack | email

Send a completion notification. Slack requires `SLACK_WEBHOOK_URL` in `.env`.

```
python main.py /Volumes/home --base-dir /organized --notify slack
```

--execute

Apply the planned file operations after an interactive y/N confirmation. Without this flag every run is a dry run.

--write-metadata

During execution, write extracted metadata alongside the organized files.

8. Common Recipes

First look at a messy volume — cheap, safe, fast:

```
python main.py /Volumes/home --base-dir /tmp/preview --max-files 500
```

Full-featured dry run on a NAS — database caching, LLM fallback, large-file handling, 8 parallel hashing threads, saved log. Safe to Ctrl+C and re-run to resume:

```
python main.py /Volumes/home --base-dir /organized \  
--use-db --llm-classify --metadata-only-size 100MB --workers 8 --dry-run-log
```

Photo library pass — images only, with metadata and AI content tags:

```
python main.py /Volumes/photos --base-dir /organized_photos \  
--use-db --file-types images --analyze-images --ai-tagging
```

Deduplicate only — report duplicates, organize just the unique files:

```
python main.py /Volumes/home --base-dir /organized \  
--use-db --skip-duplicates --duplicate-report duplicates.txt
```

The real thing — after reviewing a dry run:

```
python main.py /Volumes/home --base-dir /organized \  
--use-db --llm-classify --execute --notify slack
```

9. Project Structure

```
File_Deduplication/
+-- main.py                # CLI entry point (delegates to core/main.py)
+-- setup.py               # Packaging (console script: dedupe, extra: [ai])
+-- requirements.txt       # Core dependencies
+-- requirements-ai.txt    # Optional CLIP/torch stack
+-- Makefile               # bump / changelog / release automation
+-- README.md              # Deduplicator CLI + database reference
+-- WORKBENCH.md           # File Workbench: web UI + unified pipeline
+-- CHANGELOG.md           #
+-- config/                # YAML config + folder mapping rules
|   +-- file_type_groups.yaml
|   +-- folder_mapping.py
|   +-- folder_mappings.yaml
|   +-- image_ai_categories.yaml
|   +-- semantic_paths.yaml
+-- core/                  # Application library code
|   +-- main.py            # CLI: argument parsing and terminal output
|   +-- pipeline.py        # The pipeline as a library (CLI + server)
|   +-- parallel.py        # Per-stage execution policy
|   +-- scanner.py         # Recursive scan + atomic package detection
|   +-- hasher.py          # SHA256 hashing with DB cache
|   +-- deduplicator.py    # Duplicate detection and filtering
|   +-- classifier.py      # Rule-based classification (18+ categories)
|   +-- llm_client.py      # Local Ollama classification fallback
|   +-- organizer.py       # Folder structure planning
|   +-- previewer.py       # Dry-run previews and tree rendering
|   +-- executor.py        # Executes planned file operations
|   +-- context_detector.py # Semantic context detection (Work/Personal/...)
|   +-- ai_tagger.py       # Path-based semantic tagging
|   +-- image_analyzer.py  # EXIF/IPTC/GPS metadata extraction
|   +-- image_content_analyzer.py # CLIP-based image content analysis
|   +-- image_db.py        # Image metadata persistence
|   +-- metadata_writer.py # Metadata output during execution
|   +-- db.py              # MySQL connection and persistence helpers
+-- classify/              # Content extraction + classification ladder
|   +-- extract.py         # Any supported file -> plain text
|   +-- vision.py          # Image -> caption + OCR + EXIF
|   +-- engine.py          # Rules -> local LLM -> cloud escalation
|   +-- cloud.py           # Claude tier, with a hard spend cap
+-- search/               # RAG index + user-added metadata
|   +-- rag_store.py       # Embeddings, vector store, hybrid search
|   +-- metadata_store.py  # User-added people/tags/notes
+-- server/               # FastAPI backend for the web UI
|   +-- app.py             # Routes
|   +-- jobs.py            # Jobs in child processes, progress, cancel
|   +-- security.py        # Path guards and file allowlist
+-- web/                  # Single-page front end (no build step)
|   +-- index.html
|   +-- js/app.js
+-- database/
|   +-- schema/            # Table creation scripts (run first)
|   +-- migrations/       # Incremental schema changes (run in order)
|   +-- queries/          # Ad-hoc analysis queries
+-- docs/                 # All guides, this document, historical notes
+-- models/
|   +-- file_info.py       # FileInfo dataclass shared by the pipeline
+-- scripts/              # Automation: versioning, changelog, env setup,
```

```
|                                     # reclassification, debug tools
+-- tests/                           # Test suite (script-style + pytest-style)
+-- utils/                           # Cache, GUI, notifications, path metadata,
                                     # file type filtering
```

The flow of a file through the code: `scanner.py` → `hasher.py` → `deduplicator.py` → `classifier.py` (with `llm_client.py` as fallback) → `organizer.py` → `previewer.py` → `executor.py`. The `FileInfo` dataclass in `models/file_info.py` is the shared record passed between every stage.

10. Version Management and Releases

The project version lives in `scripts/version.yaml` and is read by `scripts/read_version.py`. `setup.py` carries the same version for packaging.

10.1 Bumping the Version

```
make bump                # bump the patch version (currently 0.10.0),
                          # commit, and push

# What it runs under the hood:
python scripts/bump_version.py patch
git add scripts/version.yaml
git commit -m "Bump patch version"
git push origin main
```

10.2 Updating the Changelog

```
make changelog           # regenerate from commit history:
                          #   scripts/gen_changelog.py > docs/CHANGELOG_LAST.md
                          #   scripts/gen_changelog.py >> CHANGELOG.md
                          #   then commits and pushes
```

10.3 Cutting a Release

```
make release             # 1. verifies no staged file exceeds 100MB
                          # 2. commits with the current version number
                          # 3. pushes to origin/main
                          # 4. creates a GitHub release via `gh release create`
                          #    using docs/CHANGELOG_LAST.md as the notes
```

Typical release sequence: **make bump** → **make changelog** → **make release**. The GitHub CLI (gh) must be installed and authenticated for the release step.

11. Git Workflow: Committing and Pushing

The repository is hosted on **GitHub** at `github.com/phe1getar/File_Deduplication`, with `main` as the working branch.

11.1 Day-to-Day Commits

```
git status                # review what changed
git add <files>           # stage specific files (or -A for all)
git commit -m "feat(scope): short description"
git push origin main
```

Commit messages follow the conventional-commits style used throughout the history: a type prefix (feat, fix, docs, chore, refactor), an optional scope in parentheses, and a concise imperative summary.

11.2 The Pre-Commit Size Guard

A pre-commit hook (symlinked to `scripts/validate_large_files.sh`) blocks any commit containing a staged file over 100MB — GitHub's hard limit. If a commit is rejected, remove the offending file from staging; generated artifacts belong in `output/`, which is gitignored.

11.3 What Never Gets Committed

- `.env` — contains database credentials
- `output/` — dry-run previews and reports
- `.venv*/` — virtual environments
- `.file_dedup_cache.json` — local hash cache
- `__pycache__/, .DS_Store`, IDE folders

All of these are covered by `.gitignore`; if git ever shows them as untracked changes, do not force-add them.

12. Tests and Utility Scripts

12.1 Script-Style Tests (run directly)

```
python tests/test_llm_classifier.py      # LLM fallback (skips if Ollama is down)
python tests/test_atomic_packages.py    # atomic .app/.pkg detection
python tests/test_db_connection.py      # MySQL connectivity + .env sanity
python tests/test_context_detection.py   # semantic context rules
python tests/test_folder_mapping.py     # custom folder mapping rules
python tests/test_image_metadata.py     # image metadata extraction
```

12.2 Pytest-Style Tests

```
pip install pytest
pytest tests/test_scanner.py tests/test_hasher.py \
       tests/test_classifier.py tests/test_organizer.py tests/test_executor.py
```

12.3 Utility Scripts

```
python scripts/reclassify_files.py      # re-classify files already in the DB
                                         # (after classifier rule changes)
python scripts/debug_classification.py    # inspect why a file classifies as it does
./scripts/setup_env.sh                  # bootstrap a starter .env file
python scripts/generate_user_guide.py    # regenerate this PDF
```

13. Troubleshooting

Symptom	Likely Cause and Fix
Web UI: 'Refusing to scan a system directory'	The source path is under a protected root (/System, /usr, /bin, ...). Point the scan at a specific user directory instead.
Web UI: a stage reports 'serial' instead of a worker pool	Expected on cheap per-file work. A pool is only built when a measured sample shows the per-item cost justifies it; forcing one would be slower. Override with WORKBENCH__WORKERS if you disagree.
Local-LLM stage is slow no matter how many cores are free	It is bounded by the Ollama server, not by core count. Set OLLAMA_NUM_PARALLEL (e.g. 8) and restart Ollama.
--cloud-classify appears to do nothing	The tier disables itself without ANTHROPIC_API_KEY (or an `ant auth login` profile). It is also skipped outright when the pre-flight estimate exceeds --cloud-cost-limit; raise the cap or narrow the run.
Server will not start: port already in use	It selects the first free port from 8000 automatically. Pass --port to start searching elsewhere.
Database initialization fails at startup	Missing or wrong DB_* values in .env. Verify with: python tests/test_db_connection.py
'Ollama server not reachable' warning	Start the server with `ollama serve` and confirm the model exists (ollama list). Override host/model via OLLAMA_HOST / LLM_MODEL.
'AI tagging requires additional libraries'	Install the optional stack: pip install -r requirements-ai.txt
PySimpleGUI install or import errors	PySimpleGUI moved to a private index in 2024. Install with: pip install --extra-index-url https://PySimpleGUI.net/install PySimpleGUI
Commit rejected: file exceeds 100MB	The pre-commit size guard fired. Unstage the file; large artifacts belong in output/ (gitignored).
Run is slow on huge video collections	Use --metadata-only-size (e.g. 1GB) to skip hashing large files, --workers 8 to parallelize network reads, and --use-db so hashes are cached across runs.
Scan phase seems silent or hung	The walk logs a heartbeat every 10 seconds (files matched, directories visited, current location). If heartbeats stop advancing, check the directory named in the last one — likely a dead mount or permission stall.
Run was interrupted (Ctrl+C, crash, reboot)	With --use-db, completed work is already saved. Re-run the same command; unchanged files show '(cached)' and are skipped instantly.
Database dies in the middle of a run	A circuit breaker trips after 3 consecutive DB failures: one loud error, then the run continues without persistence (dry runs finish normally). --execute is refused, and an in-progress execution stops cleanly before its next file move. Restore the DB and re-run.
Everything classifies as 'code' or 'application'	The path contains a structure-preserving directory name (src, scripts, software, adobe, ...). This is by design: those trees are preserved as-is.

For deeper dives, see the topic guides in docs/: duplicate detection, atomic packages, image analysis, path metadata, reclassification, database troubleshooting, and more.